Padrino's unit tests need to change the configuration of the application's logger. Instead of going through the trouble of creating and configuring a new logger, the following tests (written with the RSpec test gem) just pry open the existing application logger and change its configuration with a Context Probe:

gems/padrino-core-0.11.3/test/test_logger.rb

```
describe "PadrinoLogger" do
  context 'for logger functionality' do
    context "static asset logging" do
      should 'not log static assets by default' do
        # ...
        get "/images/something.png"
        assert_equal "Foo", body
        assert_match "", Padrino.logger.log.string
      end

      should 'allow turning on static assets logging' do
        Padrino.logger.instance_eval{ @log_static = true }
        # ...
        get "/images/something.png"
        assert_equal "Foo", body
        assert_match /GET/, Padrino.logger.log.string
        Padrino.logger.instance_eval{ @log_static = false }
      end
    end

    # ...
```

The first test accesses a static file and checks that the logger doesn't log anything. This is Padrino's default behavior. The second test uses instance_eval to change the logger's configuration and enable static file logging. Then it accesses the same URL as the first test and checks that the logger actually logged something. Before exiting, the second test resets static file logging to the default false state.

You can easily criticize these tests for being fragile: if the implementation of Logger changes and the @log_static instance variable disappears, then the test will break. Like many other things in Ruby, encapsulation is a flexible tool that you can choose to ignore, and it's up to you to decide if and when to accept that risk. The authors of Padrino decided that a quick hack inside the logger object was an acceptable workaround in this case.

Clean Rooms

Sometimes you create an object just to evaluate blocks inside it. An object like that can be called a *Clean Room*:

Spell: Clean Room